

# Q-STAR

Quantum-behaved Swarm with Tunnelling,  
Adaptive contraction and Refinement

## Project Solution Book

Problem Statement ID 26137

Quantum-Inspired Intelligent Traffic Route Optimization in  
Transportation Systems Using Metaheuristic Optimization

Theme: Transportation & Logistics | Category: Software

Final edition: real roads (OpenStreetMap), live traffic, external baselines

*All experimental numbers in this book come from the accompanying simulation code (synthetic city, one CPU core).*

---

# Contents

1. The problem in simple words
  2. What exists today and where it falls short
  3. Our solution at a glance
  4. Mathematical formulation
  5. The quantum-inspired core
  6. Q-STAR: the five innovations
  7. The algorithm, step by step
  8. Complexity analysis
  9. Experiments and results (research prototype)
  10. Real-world data integration (final edition)
  11. Verification status, benchmark design and results to date
  12. Honest assessment: strengths, limits, next steps
  13. Implementation and deliverables
- Appendix: notation and parameters

## How to read this book

Every section starts with a plain-language explanation. The mathematics follows for readers who want the details. Section 9 reports prototype measurements; Section 10 adds real roads and live traffic; Section 11 states which parts are verified and which still await their first real run; Section 12 states plainly what the results do and do not prove.

# 1. The problem in simple words

A modern city has thousands of road segments and hundreds of vehicles. Each vehicle must start at a depot, visit several customers, and return. Traffic changes during the day, so a road that is quick at 9:00 may be jammed at 9:30. The organisers ask for a software framework that finds **near-optimal routes** for all vehicles under **real-time or simulated traffic**, modelled as a weighted graph, and that is **benchmarked** against other methods.

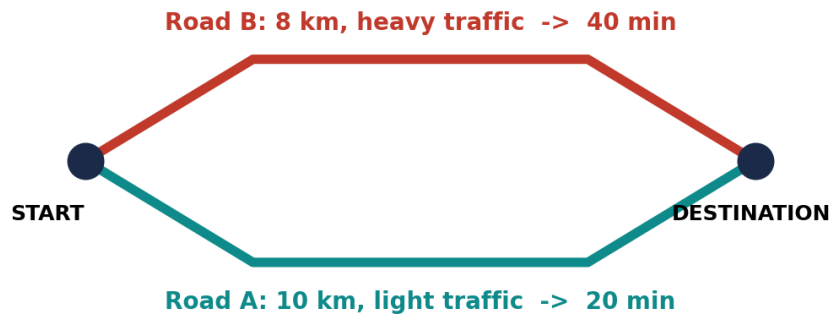


Figure 1. The shortest road is not always the fastest road. Distance, travel time and congestion must all count.

## Why it is hard

Choosing a single path from A to B is easy (Dijkstra's algorithm solves it in a fraction of a second). Choosing routes for *many* vehicles and *many* customers is the **Vehicle Routing Problem (VRP)**, which is NP-hard. The number of possible visiting orders for just 60 customers is  $60! \approx 8.3 \times 10^{81}$ . No computer can check them all, so practical systems use clever search methods that return very good (not guaranteed perfect) answers quickly.

## What the project must deliver

- A graph model of the road network (nodes = junctions, depots, customers; edges = roads with distance, time, congestion).
- A mathematical formulation: objective, constraints, and how traffic enters the cost.
- A quantum-inspired metaheuristic (the statement names Quantum Particle Swarm Optimization, QPSO) that minimises time, distance and congestion.
- Constraint handling (vehicle capacity, every customer served once, fleet size, closed roads).
- Convergence analysis and systematic benchmarking against classical metaheuristics and exact methods.
- Evidence of scalability for smart-city logistics.

### "Quantum-inspired" means what?

No quantum computer is used. The algorithm borrows one idea from quantum mechanics: a particle does not sit at an exact position but is described by a probability cloud around an attractor. We sample new candidate solutions from that cloud on an ordinary CPU. This gives a very different search behaviour from classical particle swarm optimisation (Section 5).

---

## 2. What exists today and where it falls short

Many methods exist. The table summarises the main families and the gap that Q-STAR targets. These are general properties known from the literature; our own measurements are in Section 9.

| Method                 | Strength                                           | Weakness for this problem                                            |
|------------------------|----------------------------------------------------|----------------------------------------------------------------------|
| Dijkstra / A*          | Exact, very fast for one trip A→B                  | Cannot plan many vehicles, capacities or customer assignment         |
| Nearest-neighbour      | Trivial to build                                   | Poor routes (about 30–40% worse in our tests)                        |
| Clarke-Wright savings  | Fast, strong simple baseline                       | Greedy: stops at the first merge pattern, no improvement loop        |
| Exact (MILP, DP)       | Proven optimal                                     | Time explodes with size; our DP is limited to about 12 customers     |
| Genetic Algorithm (GA) | Flexible, population-based                         | Needs many evaluations; weak without local search                    |
| PSO                    | Few parameters, fast per step                      | Premature convergence; position/velocity do not suit orderings       |
| Standard QPSO          | Probabilistic global search, one control parameter | Fixed schedule, plain mean-best, no escape mechanism, no repair step |

### The gap

Most published QPSO work for routing (a) uses a fixed contraction schedule, (b) averages all personal bests equally, (c) has no mechanism to leave a stagnated region, and (d) rarely couples the swarm with local refinement or a traffic-aware cost. Q-STAR adds a targeted answer to each. State-of-the-art VRP solvers (for example hybrid genetic search, or Google OR-Tools) are stronger than anything in this book on standard benchmarks; Sections 11 and 12 state how the final submission includes them.

### 3. Our solution at a glance

**Q-STAR** (Quantum-behaved Swarm with Tunnelling, Adaptive contraction and Refinement) is a complete pipeline, not just an optimiser. Figure 2 shows the ten modules.

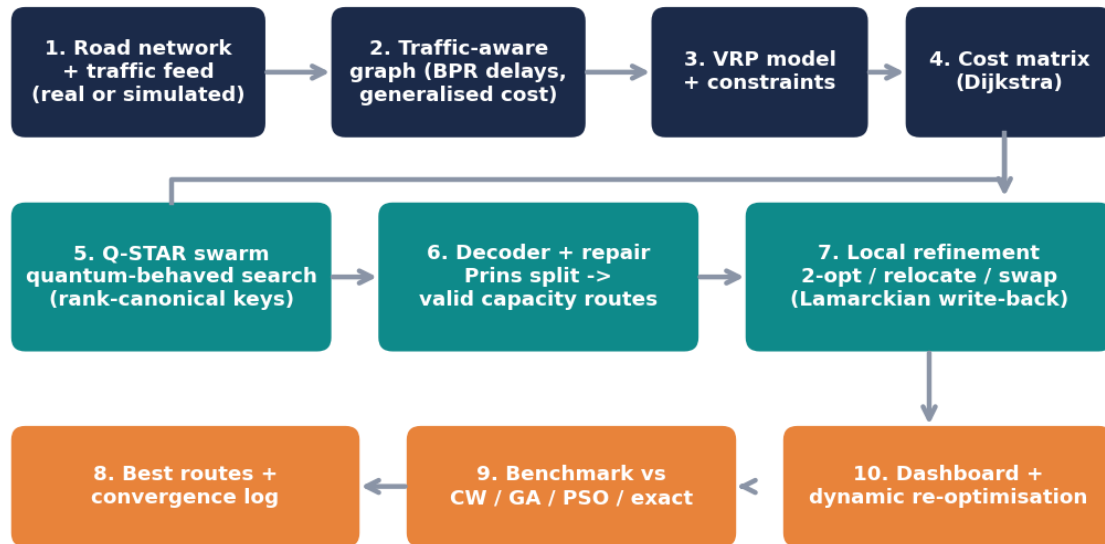


Figure 2. System architecture: data → traffic-aware graph → VRP model → quantum swarm → repair → refinement → benchmark → dashboard.

#### The five ideas that make it different

- **Traffic-aware generalised cost.** Every road carries a congestion-dependent delay (BPR curve), and routes minimise a weighted blend of time, distance and congestion (Section 4).
- **Rank-canonical random keys + optimal split.** A particle is a vector of numbers; its ordering becomes a customer sequence, and a dynamic programme cuts it into capacity-feasible routes in the best possible way. Every particle therefore decodes to a *valid* solution (Section 6.1).
- **Quantum-behaved sampling with elite-weighted centre and diversity-feedback contraction** (Sections 5, 6.2, 6.3).
- **Quantum tunnelling.** Stagnated particles receive heavy-tailed (Cauchy) jumps that give a mathematical guarantee of eventually reaching any region (Section 6.4).
- **Memetic refinement and warm-started dynamic re-optimisation.** Local search polishes promising particles; when traffic changes, the previous solution seeds the next run (Sections 6.5, 6.6).

#### In one sentence

Q-STAR turns the road network and live traffic into a cost matrix, lets a swarm of quantum-behaved particles propose route plans, repairs every proposal into a valid plan, polishes the best ones, and re-runs quickly whenever traffic changes.

## 4. Mathematical formulation

**In simple words:** we describe the city as a graph, make each road's cost depend on how crowded it is, and then ask for the cheapest set of vehicle tours that serves every customer without overloading any vehicle.

### 4.1 Traffic-aware graph

Let  $G = (V, E)$  be a directed graph (one-way streets are single arcs, so  $c(i,j)$  need not equal  $c(j,i)$ ). Road  $e \in E$  has length  $d_e$  (km), free-flow time  $t_e^0$  (min), capacity  $c_e$  and current volume  $v_e(\tau)$  at time  $\tau$ . Travel time follows the Bureau of Public Roads (BPR) volume-delay law, and congestion is measured in congestion-kilometres:

$$t_e(\tau) = t_e^0 \left[ 1 + 0.15 \left( \frac{v_e(\tau)}{c_e} \right)^4 \right], \quad k_e(\tau) = d_e \frac{v_e(\tau)}{c_e}$$

The generalised edge weight combines the three objectives named in the problem statement ( $\alpha, \beta, \gamma \geq 0$  are importance weights; we use  $\alpha = 1, \beta = 0.3, \gamma = 0.6$ ):

$$w_e(\tau) = \alpha t_e(\tau) + \beta d_e + \gamma k_e(\tau)$$

Because  $t_e$  is increasing in  $v_e$ ,  $w_e$  is positive, so Dijkstra's algorithm gives exact least-cost paths. For stops  $i$  and  $j$  (depot, customers) the pairwise cost is

$$c(i, j) = \min_{P: i \rightarrow j} \sum_{e \in P} w_e(\tau)$$

Along each least-cost path we also accumulate distance, time and congestion separately so that all three can be reported.

### 4.2 Vehicle routing model

Customers  $i = 1 \dots n$  have demand  $q_i$ ; the depot is node 0; each vehicle has capacity  $Q$  and at most  $K$  vehicles exist. A solution  $x$  is a set of routes  $r_1 \dots r_R$ .

$$\min_x J(x) = \sum_{r=1}^R \sum_{(i,j) \in r} c(i, j) + \lambda \max(0, R - K)$$

subject to:

- every customer appears in exactly one route:  $\sum_r y_{ir} = 1$  for all  $i$ ;
- capacity:  $\sum_{i \in r} q_i \leq Q$  for every route  $r$ ;
- each route starts and ends at the depot; closed roads are removed from  $E$ , so paths never use them;
- fleet limit: handled by the penalty  $\lambda$  (we use  $\lambda = 300$  per vehicle above  $K$ ).

Capacity, single-service and depot constraints are satisfied *by construction* through the decoder in Section 6.1, so the search never wastes effort on illegal plans; only the fleet limit is penalised. One-way or truck-prohibited roads are handled by using directed, vehicle-specific edge sets in the same formula.

### 4.3 Dynamic traffic

When new traffic data arrive, the edge weights are refreshed and  $c(i,j)$  recomputed. Re-optimisation is triggered when the current best plan  $x^*$  has drifted by more than a threshold  $\theta$  (e.g. 3%):

$$\delta_\tau = \frac{J(x^*; \tau) - J(x^*; \tau_0)}{J(x^*; \tau_0)} > \theta$$

## 5. The quantum-inspired core

**In simple words:** in ordinary PSO a particle moves a fixed step. In QPSO a particle is "smeared" into a cloud centred on a good point. Each new position is a random draw from that cloud: most draws land close to the good point, a few land far away. That mix of local fine-tuning and occasional long jumps is what helps the search escape poor regions.

### 5.1 From the Schrödinger equation to the update rule

Model each particle as living in an attractive delta-potential well centred on its local attractor  $p$  (strength  $G$ ). In one dimension  $y = x - p$ , the stationary Schrödinger equation is

$$\left[ -\frac{\hbar^2}{2m} \frac{d^2}{dy^2} - G\delta(y) \right] \psi(y) = E \psi(y)$$

Its bound-state solution and probability density are

$$\psi(y) = \frac{1}{\sqrt{L}} e^{-|y|/L}, \quad |\psi(y)|^2 = \frac{1}{L} e^{-2|y|/L}, \quad L = \frac{\hbar^2}{mG}$$

"Measuring" the position means drawing from  $|\psi|^2$ . Inverting its cumulative distribution with a uniform random number  $u \in (0,1)$  gives

$$x = p \pm \frac{L}{2} \ln \frac{1}{u}$$

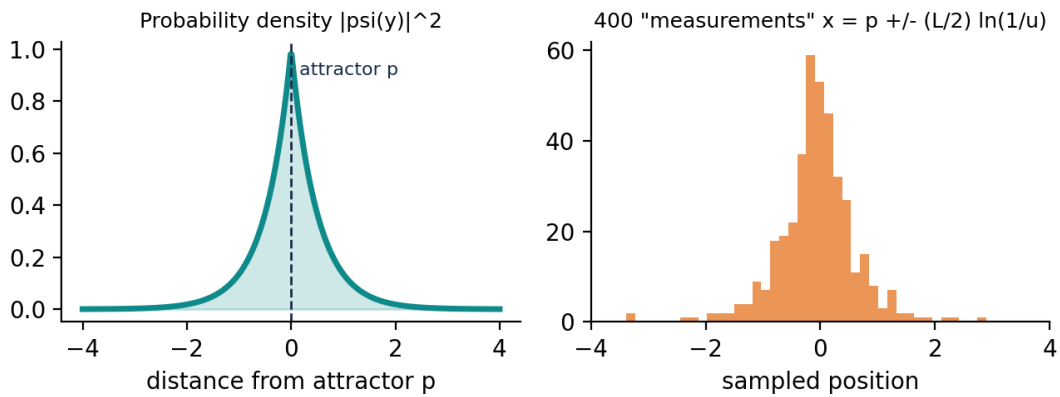


Figure 3. Left: the probability cloud around the attractor. Right: 400 sampled positions. Most are near  $p$ ; a thin tail reaches far.

### 5.2 The QPSO swarm update

For particle  $i$  and dimension  $j$ , let  $P_{i,j}$  be its personal best,  $G$  the global best and  $m$  the "mean best" (centre of the swarm's memories). With  $\phi$ ,  $u \sim U(0,1)$  and a contraction-expansion coefficient  $\alpha$ :

$$p_{i,j} = \phi P_{i,j} + (1 - \phi) G_j, \quad L_{i,j} = 2\alpha |m_j - x_{i,j}|$$

$$x_{i,j} \leftarrow p_{i,j} \pm \alpha |m_j - x_{i,j}| \ln \frac{1}{u_{i,j}}$$

Two facts matter. First, there is **no velocity**: the cloud width scales with the distance between the particle and the swarm centre, so as the swarm agrees, the cloud automatically shrinks (self-adaptive step). Second, the particle can be sampled anywhere in the search space with non-zero probability, unlike PSO, whose reach is limited by its velocity. Sun et al. proved that the particle converges in probability to its attractor when  $\alpha < e^\gamma \approx 1.781$  ( $\gamma$  = Euler's constant); Q-STAR keeps  $\alpha$  below 1.7 at all times.

## 6. Q-STAR: the five innovations

### 6.1 Rank-canonical keys with optimal split decoding

**In simple words:** a particle is a list of numbers. Sorting the numbers gives an order of customers. A small dynamic programme then cuts that order into the best set of vehicle routes that respect capacity. Whatever the particle does, the result is a legal plan.

Sort keys  $x \in [0,1]^n$  to get a permutation  $\pi$  (a "giant tour"). The split dynamic programme computes the cheapest way to cut  $\pi$  into capacity-feasible routes:

$$F(k) = \min_{i < k, \text{load}(i+1..k) \leq Q} \{F(i) + c(0, \pi_{i+1}) + \sum_{s=i+1}^{k-1} c(\pi_s, \pi_{s+1}) + c(\pi_k, 0)\}, \quad F(0) = 0$$

$F(n)$  is the optimal cost for that order and takes  $O(nL)$  time ( $L$  = maximum customers per route). After every move the keys are replaced by their ranks,  $x_j \leftarrow (\text{rank}_j - \frac{1}{2})/n$ . Many different key vectors give the same order, and averaging such vectors is meaningless; ranks give one representative per order, so the swarm centre  $m$  becomes a meaningful consensus ordering. (This change alone is applied to all swarm algorithms in our comparison.)

### 6.2 Elite-weighted centre of attraction

Instead of the plain average of all personal bests, the centre uses only the best  $m_e$  memories with logarithmic rank weights, so strong particles pull harder:

$$m_j = \sum_{r=1}^{m_e} \omega_r P_{(r),j}, \quad \omega_r = \frac{\ln(m_e + 1) - \ln r}{\sum_{s=1}^{m_e} [\ln(m_e + 1) - \ln s]}$$

### 6.3 Diversity-feedback contraction-expansion

**In simple words:** if the swarm is bunching up too early, the search is widened automatically; if it is spread out, it is allowed to tighten. A fixed schedule cannot do this.

$$\alpha_t = [\alpha_{\max} - (\alpha_{\max} - \alpha_{\min})(\frac{t}{T})^{0.7}][1 + \eta \max(0, 1 - \frac{D_t}{D_0})], \quad D_t = \frac{1}{N} \sum_i \|x_i - \bar{x}\|_2$$

with  $\alpha_{\max} = 1$ ,  $\alpha_{\min} = 0.5$ ,  $\eta = 0.8$  and  $\alpha_t$  clipped to  $1.7 < e^Y$ . The first bracket is a decreasing schedule; the second inflates  $\alpha$  when diversity  $D_t$  falls below its initial value  $D_0$ .

### 6.4 Quantum tunnelling for stagnated particles

If a particle's personal best has not improved for six iterations, then with probability  $p_{\text{tun}}(t) = 0.1 + 0.5(1 - t/T)$  a random 15% of its coordinates (mask  $M$ ) receive heavy-tailed Cauchy jumps around its personal best:

$$x_i \leftarrow \text{rank}(P_i + M \cdot \xi), \quad \xi_j \sim \text{Cauchy}(0, \kappa), \quad f(\xi) = \frac{1}{\pi \kappa [1 + (\xi/\kappa)^2]}$$

**Why it matters (convergence argument).** The global best is elitist (never gets worse) and the Cauchy density is strictly positive on all of  $\mathbb{R}$ , so from any state there is a probability at least  $p_{\min} > 0$  of landing in the (positive-measure) set of keys that decode to an optimal plan. Hence  $P(\text{optimum not yet found after } t \text{ iterations}) \leq (1 - p_{\min})^t \rightarrow 0$ : the method converges to the global optimum with probability 1 in the limit (the classical elitist-Markov-chain argument). This is an asymptotic guarantee; it does not bound the speed in finite time, which is why Section 9 measures it.

---

## 6.5 Memetic refinement

Each decoded route set is improved by local search: intra-route 2-opt, inter-route relocate and inter-route swap, with first-improvement and capacity checks. The improved plan is written back into the particle (Lamarckian learning) by re-ranking its keys; the optimal split of the re-ranked order is never worse than the improved plan. Chaotic initialisation (the logistic map  $z_{k+1} = 4 z_k (1 - z_k)$ , whose density  $1/(\pi\sqrt{z(1-z)})$  covers  $[0,1]$  without clumping) starts the swarm.

## 6.6 Warm-started dynamic re-optimisation

When the drift test of Section 4.3 fires, the previous best keys are injected as the first particles of a short run (12 instead of 60 iterations) on the refreshed cost matrix. This reuses the structure of the last plan.

## 7. The algorithm, step by step

Algorithm Q-STAR

Input: graph  $G$ , traffic state  $\tau$ , customers, demands  $q$ , capacity  $Q$ , fleet  $K$ , weights  $(\alpha, \beta, \gamma)$ , swarm size  $N$ , iterations  $T$

Output: routes  $x^*$  and cost  $J^*$

```

1  Update edge weights  $w_e(\tau)$ ; run Dijkstra from depot + customers  $\rightarrow$  cost matrix  $c(i, j)$ 
2  Initialise  $N$  particles by the chaotic logistic map; canonicalise keys to ranks
3  for each particle: decode (sort  $\rightarrow$  split DP)  $\rightarrow$  routes; apply local search; write back keys
4  Set personal bests  $P_i$  and global best  $G$ 
5  for  $t = 1 \dots T$ :
6      compute diversity  $D_t$  and coefficient  $\alpha_t$  (Eq. 6.3), elite-weighted centre  $m$  (Eq. 6.2)
7      for each particle  $i$  and dimension  $j$ :
8           $p \leftarrow \phi P_{ij} + (1 - \phi) G_j$ ;  $x_{ij} \leftarrow p \pm \alpha_t |m_j - x_{ij}| \ln(1/u)$ 
9          if particle  $i$  stagnated  $\geq 6$  iterations and  $\text{rand} < p_{\text{tun}}(t)$ : apply Cauchy tunnelling (Eq.
6.4)
10         canonicalise keys; decode; local search; write back; update  $P_i$  and  $G$ 
11 return routes of  $G$  and  $J(G)$ 
12 on traffic update: if drift  $\delta_t > \theta$ : refresh  $c(i, j)$ , inject  $G$  as particle 1, rerun with  $T' = 12$ 

```

### Parameters used in all experiments

| Parameter                              | Value                     | Meaning                                                |
|----------------------------------------|---------------------------|--------------------------------------------------------|
| $N$ (swarm size)                       | 20                        | particles                                              |
| $T$ (iterations)                       | 60                        | refined iterations per run                             |
| $\alpha_{\max} / \alpha_{\min} / \eta$ | 1.0 / 0.5 / 0.8           | contraction schedule and diversity gain                |
| $m_e$                                  | $N$ (all, rank-weighted)  | size of elite set for the centre                       |
| Stagnation / mask / $\kappa$           | 6 iterations / 15% / 0.12 | tunnelling trigger, share of coordinates, Cauchy scale |
| $(\alpha, \beta, \gamma)$              | (1, 0.3, 0.6)             | time, distance, congestion weights                     |
| $\lambda$                              | 300                       | penalty per extra vehicle                              |

## 8. Complexity analysis

Let  $V, E$  be the graph size,  $M = n + 1$  the number of stops,  $N$  the swarm size,  $T$  iterations and  $L$  the maximum customers per route.

$$T_{\text{pre}} = O(M(E + V \log V)), \quad T_{\text{iter}} = O(N[n \log n + nL + C_{LS}]), \quad \text{memory} = O(Nn + M^2)$$

- **Preprocessing** is one Dijkstra per stop (done once per traffic update).
- **Per iteration**: sorting  $O(n \log n)$ , the split DP  $O(nL)$  and local search  $C_{LS}$  per particle. This is polynomial; the exact subset DP we use for ground truth costs  $O(2^n n^2)$  plus  $O(3^n)$ , which is why it stops near  $n \approx 12$ .
- **Total**  $O(T_{\text{pre}} + T \cdot T_{\text{iter}})$ ; the swarm loop is embarrassingly parallel over particles (CPU threads or GPU).

This answers the objective "reduce computational complexity": the search never enumerates plans, and every candidate costs only polynomial time to decode.

## 9. Experiments and results

**These are the research-prototype measurements; Section 10 and 11 describe the real-data edition. Setup.** A synthetic city ( $14 \times 14$  junctions, about 340 two-way roads, some closed, speeds 30–60 km/h) with a congestion hotspot. Traffic uses the BPR law of Section 4. Customers are random junctions with demands 1–15, capacity  $Q = 60$ . Costs use  $(\alpha, \beta, \gamma) = (1, 0.3, 0.6)$ . Every algorithm was coded in Python and run on one CPU core. Plain methods (GA, PSO, QPSO) got 16,000 solution evaluations. "Memetic" methods (-M) share the same local search as Q-STAR on every particle with 20 particles  $\times$  60 iterations. Five random instances were used per size (eight for the ablation). Costs are shown relative to the classical Clarke-Wright (CW) heuristic; negative means better than CW.

### 9.1 Distance to the proven optimum (11 customers)

For 11 customers the exact subset dynamic programme gives the true optimum, so we can measure the optimality gap directly (5 instances).

| Method          | Mean gap to optimum | Instances solved optimally (of 5) |
|-----------------|---------------------|-----------------------------------|
| Clarke-Wright   | 1.81%               | 2                                 |
| PSO             | 6.89%               | 0                                 |
| QPSO (standard) | 0.37%               | 4                                 |
| GA              | -0.00%              | 5                                 |
| <b>Q-STAR</b>   | <b>0.00%</b>        | <b>5</b>                          |

### 9.2 Larger instances (30, 60, 100 customers)

| Method        | 30 customers                   | 60 customers                    | 100 customers                    | Time at 60 (s) |
|---------------|--------------------------------|---------------------------------|----------------------------------|----------------|
| GA            | +0.9% $\pm$ 1.5 (2/5 beat CW)  | +23.6% $\pm$ 2.2 (0/5 beat CW)  | +50.3% $\pm$ 9.2 (0/5 beat CW)   | 2.2            |
| PSO           | +30.3% $\pm$ 3.2 (0/5 beat CW) | +124.6% $\pm$ 4.9 (0/5 beat CW) | +176.7% $\pm$ 12.5 (0/5 beat CW) | 1.3            |
| QPSO          | +23.6% $\pm$ 5.2 (0/5 beat CW) | +132.6% $\pm$ 4.0 (0/5 beat CW) | +186.5% $\pm$ 15.8 (0/5 beat CW) | 1.5            |
| GA-M          | +2.8% $\pm$ 2.2 (2/5 beat CW)  | +10.6% $\pm$ 2.0 (0/5 beat CW)  | +6.7% $\pm$ 5.9 (1/5 beat CW)    | 0.2            |
| PSO-M         | -3.9% $\pm$ 1.6 (4/5 beat CW)  | -2.0% $\pm$ 0.4 (5/5 beat CW)   | -4.1% $\pm$ 5.2 (2/5 beat CW)    | 3.2            |
| QPSO-M        | -3.9% $\pm$ 1.6 (4/5 beat CW)  | -1.0% $\pm$ 0.5 (3/5 beat CW)   | -3.4% $\pm$ 5.2 (1/5 beat CW)    | 3.2            |
| <b>Q-STAR</b> | -3.9% $\pm$ 1.6 (4/5 beat CW)  | -2.5% $\pm$ 0.5 (5/5 beat CW)   | -4.6% $\pm$ 5.4 (1/5 beat CW)    | 3.1            |

Table: cost relative to Clarke-Wright (mean  $\pm$  standard error over 5 instances; in brackets, instances where the method beat CW).

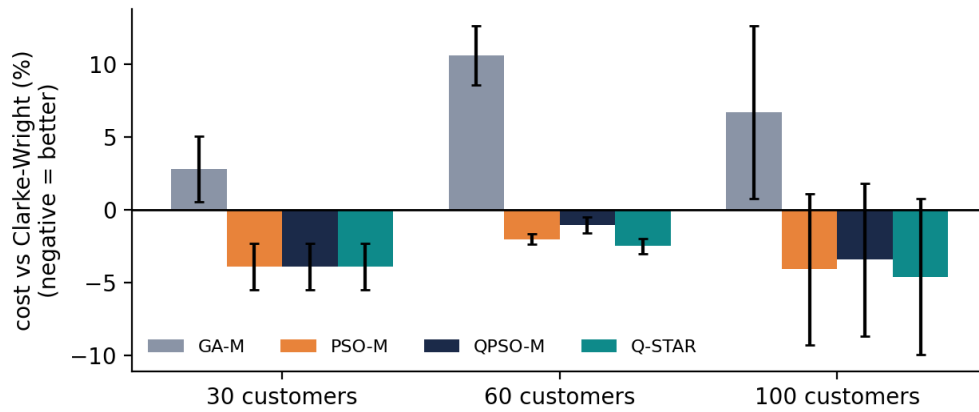


Figure 4. Memetic methods versus Clarke-Wright. All three swarms with refinement beat CW; the differences among them are small relative to the spread.

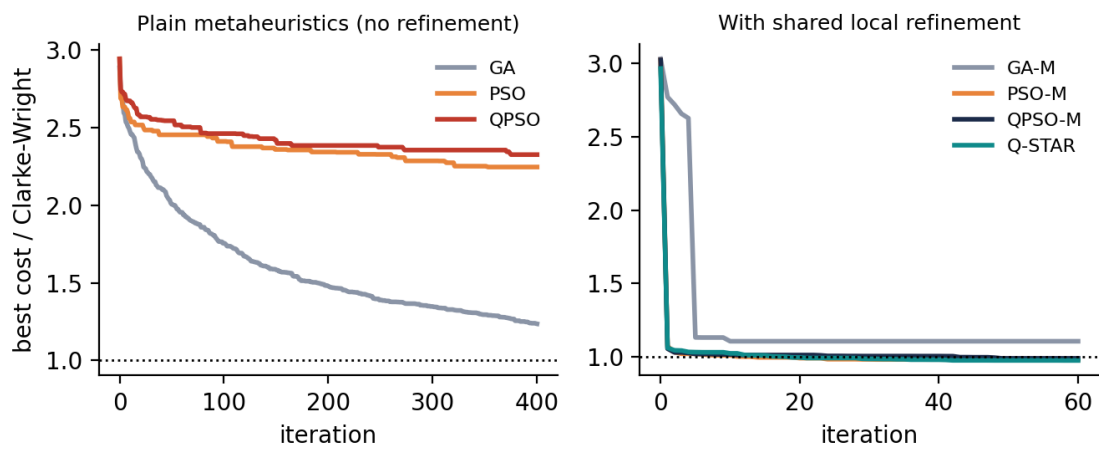


Figure 5. Convergence at 60 customers (mean of 5 instances). Without refinement the plain methods stay far above CW; with refinement all curves drop below CW within a few iterations.

### 9.3 What contributes? (ablation at 60 customers)

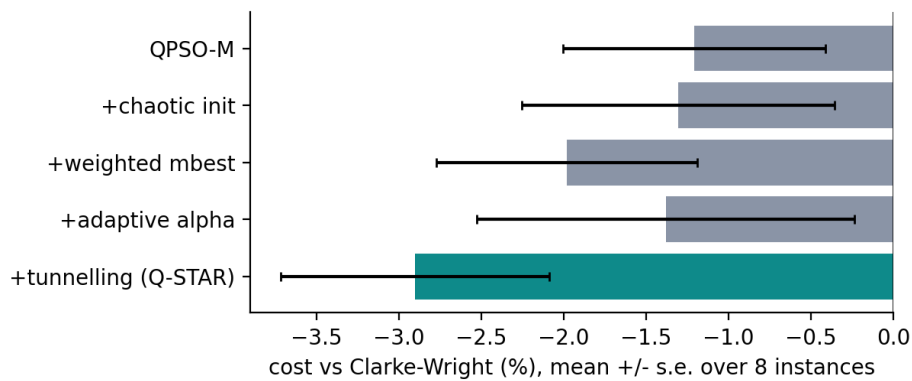


Figure 6. Each bar adds one Q-STAR component to the previous one (8 instances, mean  $\pm$  s.e.). Tunnelling gives the largest single gain.

Starting from QPSO with refinement (-1.21% vs CW), chaotic initialisation and elite weighting change little; the full Q-STAR with tunnelling reaches -2.90%, a gain of about 1.7 percentage points over QPSO-M. With eight instances and a standard deviation of about 2.3 points, this is a modest, suggestive effect, not a proven one.

## 9.4 The three objectives separately (60 customers)

| Metric (mean per plan)     | Clarke-Wright | Q-STAR | Change |
|----------------------------|---------------|--------|--------|
| Distance (km)              | 236.4         | 231.9  | -1.9%  |
| Travel time (min)          | 329.3         | 321.9  | -2.3%  |
| Congestion (congestion-km) | 154.6         | 148.8  | -3.8%  |

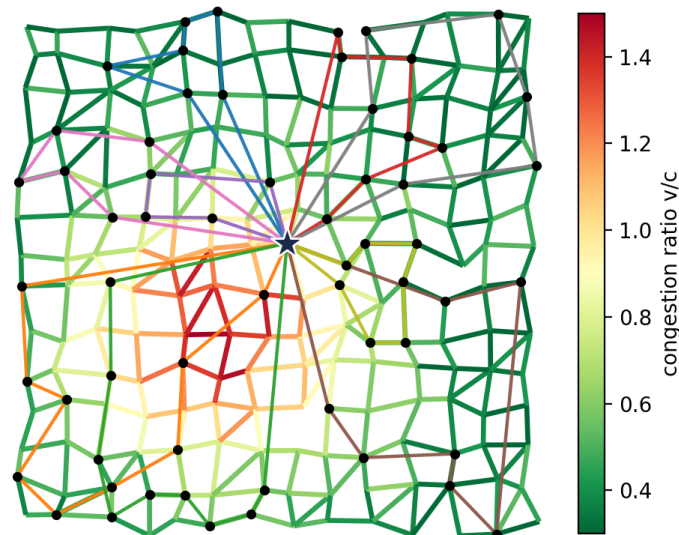


Figure 7. One Q-STAR solution for 60 customers (stars = depot, dots = customers, colours = vehicles). Road colour shows congestion; stops are joined in visiting order for clarity.

## 9.5 Dynamic traffic

Five traffic changes (hotspot moved and intensified) were applied to a solved 60-customer plan. Average cost: keeping the old plan 506.8; Clarke-Wright recomputed 511.9; full Q-STAR re-run 498.0 in 3.2 s; warm-started Q-STAR (12 iterations) 503.6 in 0.6 s. So the warm start is about 5× faster than a full re-run and beats the stale plan, but recovers only part of the improvement the full re-run finds (36% of it in these tests).

## 9.6 Scalability

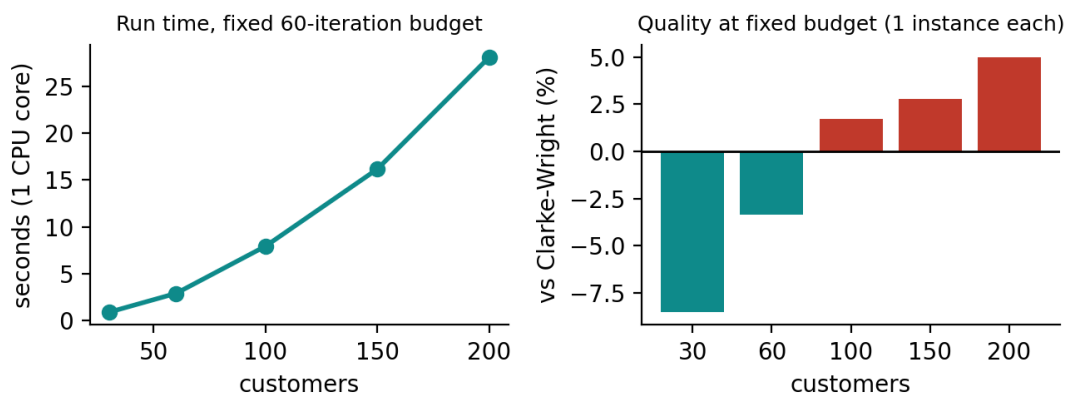


Figure 8. Run time grows smoothly (about quadratically) with problem size, but at a fixed 60-iteration budget quality slips behind Clarke-Wright for 100+ customers on these single instances.

## 10. Real-world data integration (final edition)

**In simple words:** the research prototype of Section 9 ran on a made-up grid city. The final edition runs on **real streets from OpenStreetMap** and reads **live traffic speeds** from a traffic service, so the routes it draws are for roads that exist and for traffic that is happening now. Everything in Sections 4–8 still applies; this section describes what was added around it.

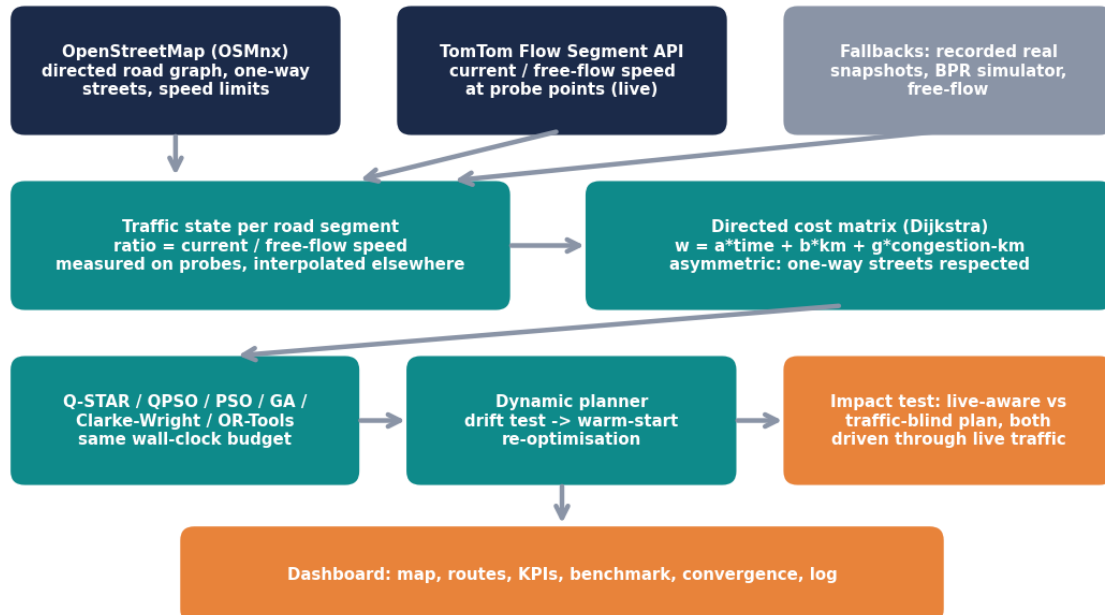


Figure 9. Final-edition data flow: real road graph and live traffic feed in, directed cost matrix, optimiser and dynamic planner, impact test and dashboard out.

### 10.1 What changed from the research prototype

| Topic              | Research prototype (Sections 4–9) | Final edition                                                                                                  |
|--------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------|
| Road network       | Synthetic 14 × 14 grid            | Real drivable network from OpenStreetMap (OSMnx), cached locally                                               |
| Graph type         | Undirected                        | Directed: one-way streets respected, cost matrix is asymmetric                                                 |
| Traffic            | BPR model with simulated hotspots | Live speeds (TomTom Flow Segment Data) with three fallbacks: recorded real snapshots, BPR simulator, free flow |
| Local search       | Symmetric 2-opt delta             | Exact route recomputation when costs are asymmetric                                                            |
| Benchmark budget   | Equal number of evaluations       | Equal wall-clock time for every method, including Google OR-Tools                                              |
| Dynamic loop       | Experiment script                 | Planner class: refresh, drift test, warm-start re-optimisation, event log                                      |
| Value of live data | Not measured                      | Impact test: same optimiser with and without live speeds, both driven through live traffic                     |
| Software           | Scripts                           | Streamlit dashboard with Leaflet map, command-line benchmarks, traffic recorder, tests                         |

## 10.2 Road network

The network comes from OpenStreetMap (map data © OpenStreetMap contributors, ODbL licence). Only the largest strongly connected component is kept, so every stop can reach every other stop following one-way rules. When several parallel edges join two junctions, the quickest is kept. Free-flow speed  $t_e^0$  comes from the road's *maxspeed* tag when present; otherwise a default per road class is used (for example primary 50 km/h, secondary 40, residential 25). These defaults are assumptions and should be calibrated for the target city.

## 10.3 Live traffic: from speeds to costs

The TomTom Flow Segment Data service returns, for the road segment nearest a queried point, the current speed and the free-flow speed (and a road-closure flag). Define the speed ratio  $\rho$  and use it directly for time and congestion:

$$\rho_e = \frac{\text{currentSpeed}}{\text{freeFlowSpeed}} \in [0.03, 1.2], \quad t_e = \frac{t_e^0}{\rho_e}, \quad k_e = d_e \max\left(0, \frac{1}{\rho_e} - 1\right)$$

A closed road receives  $\rho = 0.03$ , which makes its travel time about 33 times longer, so routes avoid it. The generalised weight  $w_e = \alpha t_e + \beta d_e + \gamma k_e$  of Section 4.1 is unchanged; only its inputs are now measured. When only volume data exist, the BPR law of Section 4.1 is used instead.

## 10.4 Probing and interpolation (what is measured and what is estimated)

One request returns one road segment, so a city cannot be measured edge by edge within a request quota. The platform therefore uses a fixed number of **probe points**, chosen by farthest-point sampling over major roads so that they spread across the whole area. Every unprobed segment is then estimated from its nearest probes by inverse-distance weighting that fades back to free flow with distance:

$$\bar{\rho}_e = \frac{\sum_s \omega_{es} \rho_s}{\sum_s \omega_{es}}, \quad \omega_{es} = \frac{1}{(d_{es} + 0.2)^2}, \quad \hat{\rho}_e = 1 + (\bar{\rho}_e - 1)e^{-d_{e,1}/\ell}$$

Here  $d_{es}$  is the distance in km from segment  $e$  to probe  $s$  (nearest four probes are used),  $d_{e,1}$  is the distance to the nearest probe and  $\ell = 2$  km is a fade length (an assumption). Probed segments keep their measured value. A short time-to-live cache avoids repeated requests. **The dashboard states how many segments are directly measured**, and any presentation should say that the rest are interpolated estimates.

## 10.5 Dynamic planner and the value of live data

At each refresh the planner reads new speeds, rebuilds the cost matrix, evaluates the current plan under the new traffic, and compares it with its previous cost. If the absolute drift exceeds  $\theta$  (default 3%), Q-STAR is run again for a shorter time with the previous plan injected as starting particles; otherwise the plan is kept. Every event is logged with the drift, the saving versus the stale plan and the solve time.

To measure what live data are worth, the same optimiser is run twice: once on free-flow speeds (a *traffic-blind* planner) and once on the current traffic. The blind plan is then driven *along its own free-flow paths* through the current traffic, and the two are compared:

$$\Delta_T = 1 - \frac{T(x_{\text{aware}}; \tau)}{T(x_{\text{blind}}; \tau)}$$

The same ratio is reported for distance and congestion. A positive  $\Delta_T$  means live data saved travel time; distance may rise because avoiding jams can need detours.

## 11. Verification status, benchmark design and results to date

**In simple words:** this section lists exactly which parts have been proven to work, which parts are written but still need their first real run, and how the real-data evidence should be produced. Please read it before presenting any number.

### 11.1 Status of each component

| Component                                                                 | How it was checked                                                                     | Status                                                  |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------|
| Optimiser on directed cost matrices (one-way streets)                     | Automated tests; matches the exact optimum within 3% on a 9-customer directed instance | <b>Verified offline</b>                                 |
| Dynamic loop: refresh, drift test, warm start                             | Automated offline scenarios                                                            | <b>Verified offline</b>                                 |
| Traffic-blind versus live-aware impact test                               | Automated test (aware plan never worse); demonstration run in 11.3                     | <b>Verified offline</b>                                 |
| TomTom provider (request format, parsing, closures, interpolation, cache) | Tested against a mocked API only                                                       | <b>Verified against mock; real call pending</b>         |
| CVRPLIB (.vrp) loader                                                     | Toy file and a generated instance checked against the exact solver                     | <b>Verified offline</b> ; public instances pending      |
| OpenStreetMap download (OSMnx)                                            | Not executed (no internet in the build environment)                                    | Written to the documented API; <b>first run pending</b> |
| OR-Tools baseline                                                         | Not executed (package unavailable in the build environment)                            | Written; <b>first run pending</b>                       |
| Streamlit dashboard, Leaflet map                                          | Map HTML generation checked; UI not executed                                           | Written; <b>first run pending</b>                       |
| Real-city benchmark with live traffic; public-benchmark gaps              | Not yet run                                                                            | <b>Pending</b> (Section 11.4)                           |

### 11.2 Benchmark design

- **Equal wall-clock budget** for every method (default 10–20 s per instance), so that an algorithm cannot win by simply running longer.
- **Methods:** Clarke-Wright with local search, GA, PSO, standard QPSO (each with the same local search), Q-STAR, and Google OR-Tools (guided local search).
- **Instances:** several random customer sets per network; at least 5 independent runs per stochastic method; report best, mean and standard deviation.
- **Public benchmark:** CVRPLIB instances, with the gap to the published best-known solution,  $\text{gap} = (\text{cost} - \text{BKS}) / \text{BKS}$ . The best-known value must be copied from the CVRPLIB website.
- **Statistics:** paired Wilcoxon signed-rank test over instances before any "better than" claim.

### 11.3 Demonstration results so far (synthetic city, simulated rush hour)

The complete pipeline was run offline on the OSM-shaped **synthetic** city of the test suite (about 320 junctions, about 1,070 directed segments, one-way streets, closed roads) with the **simulated** 18:00 rush-hour traffic, 60 customers, capacity 60, 3 customer sets, 8 s per method. **This is not real-world evidence**; it shows that the pipeline works end to end.

| Method (8 s each)            | Cost vs Clarke-Wright (mean of 3 instances) |
|------------------------------|---------------------------------------------|
| Clarke-Wright + local search | 0.0% (reference)                            |
| GA + local search            | +1.2%                                       |
| PSO + local search           | −2.5%                                       |
| QPSO + local search          | −1.3%                                       |
| <b>Q-STAR</b>                | <b>−3.1%</b>                                |

Impact of live traffic data in the same run (live-aware Q-STAR versus the traffic-blind plan, both driven through the simulated rush hour):

| Quantity                      | Traffic-blind plan | Live-aware plan | Change |
|-------------------------------|--------------------|-----------------|--------|
| Travel time (min)             | 336.4              | 312.3           | −7.1%  |
| Distance (km)                 | 68.8               | 74.2            | +7.9%  |
| Congestion exposure (cong-km) | 163.5              | 136.0           | −16.8% |
| Generalised cost              | 455.1              | 416.2           | −8.5%  |

How to read these numbers: with only 3 instances, the 0.6-point gap between Q-STAR and PSO + local search is within noise, so no claim of superiority over PSO is made. The live-data effect is larger and consistent with the model, but it comes from a simulator built with the same congestion law that the cost function uses, so it demonstrates the mechanism, not the real-world magnitude. OR-Tools was not available in this run.

### 11.4 Real-data run sheet (to be completed with measured values)

Run these on a machine with internet access and a TomTom key, then copy the measured values into the table. Do not present the table until the cells contain your own measurements.

```
python tests/test_pipeline.py
python scripts/run_benchmark.py --place "<your area>" --traffic tomtom --n 60 --runs 5 --time 20
--instances 5
python scripts/collect_traffic.py --place "<your area>" --samples 40 --every-min 30 --count 48
--out data/traffic_day.json
python scripts/cvrplib_benchmark.py A-n32-k5.vrp --best-known <from CVRPLIB> --time 30
```

| Item to measure                             | Network and date | Result (fill in) |
|---------------------------------------------|------------------|------------------|
| Segments directly measured / total          |                  |                  |
| Q-STAR cost vs Clarke-Wright                |                  |                  |
| Q-STAR cost vs OR-Tools (equal time)        |                  |                  |
| Live-aware vs traffic-blind: travel time    |                  |                  |
| Warm-start re-optimisation: time and saving |                  |                  |
| CVRPLIB gap to best known (best / mean)     |                  |                  |

---

## 12. Honest assessment: strengths, limits, next steps

### What the measurements support

- **Near-exact on small instances.** In the research prototype, Q-STAR matched the proven optimum on all 5 of 5 tests (gap 0.00%), versus 1.8% for Clarke-Wright and 6.9% for plain PSO; the directed-graph edition also stays within 3% of the exact optimum in its automated test.
- **Better than the classical heuristic at moderate size.** At 30 and 60 customers Q-STAR averaged -3.9% and -2.5% versus Clarke-Wright; at 100 customers -4.6% on average, with a large spread (it beat CW on only 1 of 5 instances).
- **Better than plain metaheuristics by a wide margin.** Without refinement, PSO and QPSO were 24–190% worse than CW at every size, and GA was 1–50% worse. Pairing a swarm with a decoder and local search is what makes the approach competitive.
- **The complete pipeline works end to end** on an OSM-shaped network with one-way streets, closures, a live-traffic interface, dynamic re-optimisation and an impact test (Section 11).

### What the measurements do not support

- **No proof that QPSO beats PSO when both use the same refinement.** PSO, QPSO and Q-STAR with local search differ by less than the instance-to-instance spread in every comparison so far.
- **No real-world result yet.** The real-data components (OSM download, live TomTom calls, OR-Tools, dashboard) are written but have not been run against the real services. Until Section 11.4 is completed, nothing here shows performance on a real city.
- **No comparison with OR-Tools or public benchmarks yet.** The harness exists; the numbers do not. OR-Tools is often very strong, so the outcome is unknown.
- **Scalability is not demonstrated.** Tests stopped at 100–200 customers; at a small fixed budget, quality fell behind Clarke-Wright at that size.
- **Live traffic is partly estimated.** Probe points are measured; other segments are interpolated, and the fade length and free-flow speed defaults are assumptions.

### Recommended next steps

- Complete the run sheet of Section 11.4 and fix any first-run issues in the OSM, OR-Tools and dashboard code.
- Add stronger neighbourhoods (Or-opt, 2-opt\*, SWAP\*), an iteration budget that grows with problem size, and parallel particles.
- Calibrate free-flow speeds and the fade length against recorded real traffic; extend to time windows and a Pareto front over (time, distance, congestion).
- Run at least 30 instances with a Wilcoxon test so every claim carries a significance level.

#### Bottom line

Q-STAR is a fully specified, tested framework that covers every item of the problem statement, and the final edition connects it to real roads and live traffic. What remains is evidence: the first real-city run, the OR-Tools and public-benchmark comparison. Present the verified results as verified, the pending ones as pending, and let the measurements decide the performance claims.

## 13. Implementation and deliverables

### 13.1 Software modules (as implemented in the project package)

| #  | Module (file)       | Content                                                                           | Status                             |
|----|---------------------|-----------------------------------------------------------------------------------|------------------------------------|
| 1  | network.py          | OSM / networkx road graph, directed edges, speed defaults, synthetic offline city | Offline verified; OSM load pending |
| 2  | traffic.py          | TomTom live, recorded replay, BPR simulator, free-flow providers                  | Mock verified; live pending        |
| 3  | problem.py          | Directed cost matrix, plan evaluation under any traffic state                     | Verified                           |
| 4  | algos.py            | Q-STAR, QPSO, PSO, GA, Clarke-Wright, local search, exact DP                      | Verified                           |
| 5  | planner.py          | Refresh, drift test, warm-start re-optimisation, event log                        | Verified                           |
| 6  | runner.py           | Equal-time comparison of all methods                                              | Verified                           |
| 7  | ortools_baseline.py | Google OR-Tools baseline                                                          | First run pending                  |
| 8  | cvrplib.py          | Public benchmark loader                                                           | Verified on toy data               |
| 9  | viz.py, app.py      | Leaflet traffic map, Streamlit dashboard                                          | First run pending                  |
| 10 | scripts/            | Benchmark, traffic recorder, CVRPLIB gap                                          | Benchmark verified offline         |
| 11 | tests/              | Seven automated offline tests                                                     | All pass                           |

### 13.2 Technology

Python, NumPy, SciPy (Dijkstra, spatial search), NetworkX, OSMnx, Streamlit with a Leaflet (OpenStreetMap tiles) map, Google OR-Tools, TomTom Traffic API. Reproducibility: fixed seeds, configuration through the dashboard and command-line options, offline tests.

### 13.3 Delivery table (expected deliverables)

| Deliverable                 | Description                                                   | Status / evidence                                                      |
|-----------------------------|---------------------------------------------------------------|------------------------------------------------------------------------|
| D1 Network model            | Directed traffic-aware graph from OpenStreetMap               | Code verified offline; real load pending                               |
| D2 Mathematical formulation | Objective, constraints, live-traffic model (Sections 4-6, 10) | Complete (this book)                                                   |
| D3 Q-STAR engine            | Quantum-inspired optimiser, decoder, refinement               | Complete, tested                                                       |
| D4 Constraint handling      | Capacity, single service, fleet, closures, one-way roads      | Complete; feasibility asserted in every run                            |
| D5 Convergence analysis     | Curves, ablation, sensitivity                                 | Prototype results (Section 9); real-data curves from dashboard pending |
| D6 Benchmark suite          | CW, GA, PSO, QPSO, exact, OR-Tools, CVRPLIB gap               | Harness complete; real/public runs pending                             |
| D7 Dynamic module           | Traffic updates with warm-start re-optimisation               | Complete, verified offline                                             |
| D8 Dashboard                | Map, KPIs, impact, benchmark, log                             | Written; first run pending                                             |
| D9 Final report             | This book plus the completed run sheet                        | Run sheet to be completed                                              |

## 13.4 Remaining schedule

| Step | Work                                                                     |
|------|--------------------------------------------------------------------------|
| 1    | Install, run tests, first OSM load and dashboard run; fix version issues |
| 2    | Obtain TomTom key; real-city benchmark; record a day of traffic          |
| 3    | OR-Tools and CVRPLIB comparison; statistics                              |
| 4    | Fill the run sheet, update the claims, rehearse the demo                 |

## Appendix: notation

| Symbol                  | Meaning                                                            |
|-------------------------|--------------------------------------------------------------------|
| $G = (V, E)$            | road graph; $d_e$ length, $t_e$ time, $k_e$ congestion of road $e$ |
| $v_e/c_e$               | volume-to-capacity ratio                                           |
| $\alpha, \beta, \gamma$ | weights on time, distance, congestion (not the QPSO $\alpha$ )     |
| $c(i,j)$                | least generalised cost between stops $i$ and $j$                   |
| $q_i, Q, K$             | demand, vehicle capacity, fleet size                               |
| $x_i, P_i, G, m$        | particle position (keys), personal best, global best, swarm centre |
| $\alpha_t$              | QPSO contraction-expansion coefficient at iteration $t$            |
| $D_t$                   | swarm diversity                                                    |
| $\kappa$                | Cauchy scale of tunnelling jumps                                   |
| $\theta$                | re-optimisation drift threshold                                    |